WORKOUT TRACKER

Product & Engineering Specification for Codex

A private Strong-inspired workout logging PWA for approximately 10 users

Area	Decision
Backend	.NET 10 / ASP.NET Core 10 Web API
Frontend	Vue 3 + TypeScript + Vite + Pinia
Database	PostgreSQL + Entity Framework Core 10
Authentication	ASP.NET Core Identity + JWT access tokens + refresh tokens
UX	Mobile-first responsive PWA, dark mode first
Target scale	~10 users; simple, secure, low-cost infrastructure
Primary goal	Fast workout logging, routines, history, PRs, charts, body tracking and Pro-style features

Version 1.0 | *August 2026*

How to use this document

This specification is designed to be given directly to Codex as the source of truth for architecture, behavior, acceptance criteria, and implementation sequence. Codex should not attempt to implement every epic in a single pass. Build in phases, keep the application runnable after each phase, and update migrations/tests as features are added.

- First give Codex the Project Guardrails and Architecture sections.
- Then implement the Foundation and MVP phases in order.
- Give Codex one epic or a small coherent group of epics per coding task.
- Require tests and database migrations in the same change as each backend feature.
- Do not merge generated work unless the app builds, tests pass, and the main mobile workflow has been manually checked.

1. Product Vision

Build a private web application inspired by the useful functionality of Strong, intended primarily for personal use and a small trusted group. It is not a pixel-for-pixel clone. The product should optimize for fast use between gym sets, progressive overload, complete workout history, useful training analytics, and ownership of the user's data.

1.1 Product principles

- Logging a set should take only a few taps.
- The previous workout must be visible while training.
- The backend is the source of truth for business rules and authorization.
- Every user owns their own data; no cross-user leakage is acceptable.
- The product remains useful on a phone with intermittent gym connectivity.
- Data must be exportable; users are never locked in.
- Advanced recommendations are suggestions, not automatic training decisions.

1.2 MVP definition

The MVP is usable when a registered user can create routines, start a routine, log sets with previous values visible, use a rest timer, finish the workout, view history, inspect exercise history, and see automatically detected PRs. Everything else builds on that stable core.

2. Technical Architecture

Layer	Technology / Responsibility
Web client	Vue 3, TypeScript, Vite, Vue Router, Pinia, Tailwind CSS, VueUse, Chart.js
API	ASP.NET Core 10 Web API, OpenAPI/Swagger, controllers or minimal APIs with consistent conventions
Application	Use cases, DTOs, validation, authorization-aware services, domain orchestration
Domain	Entities, value objects, enums, domain rules; no infrastructure dependencies
Infrastructure	EF Core 10, PostgreSQL, ASP.NET Identity, file/object storage abstractions, logging
Authentication	ASP.NET Core Identity, short-lived JWT access tokens, rotating refresh tokens
Offline	Service worker + IndexedDB outbox for active workout changes
Testing	xUnit backend tests; frontend unit/component tests; selected E2E tests for critical flows
Observability	Serilog structured logs, health endpoint, correlation IDs, safe error responses

2.1 Solution structure

```
WorkoutTracker/
  src/
    WorkoutTracker.Domain/
    WorkoutTracker.Application/
    WorkoutTracker.Infrastructure/
    WorkoutTracker.Api/
    WorkoutTracker.Web/
  tests/
    WorkoutTracker.UnitTests/
    WorkoutTracker.IntegrationTests/
    WorkoutTracker.Web.Tests/
  .github/workflows/
  docker-compose.yml
  README.md
```

2.2 Project guardrails for Codex

- Use .NET 10, nullable reference types, async I/O, dependency injection and EF Core migrations.
- Use PostgreSQL in development and production. Do not introduce SQLite-specific behavior into production code.
- Never expose EF entities directly from API endpoints; use request/response DTOs.
- All user-owned records must be filtered and authorized server-side using the authenticated UserId.
- Never trust a UserId supplied by the frontend for ownership. Resolve the current user from the authenticated principal.
- Use database constraints and indexes for data integrity in addition to application validation.
- Use TypeScript and Vue Composition API with <script setup>. Keep components small and reusable.
- Mobile-first design is mandatory. The active workout screen is the highest-priority UI.

- All schema changes require migrations. Never use EnsureCreated in production.
- Every completed epic must leave the solution building and tests passing.
- Do not silently add paid third-party dependencies. Prefer open-source packages and provider abstractions.
- Do not copy Strong trademarks, copyrighted assets, proprietary exercise media, or exact visual designs.

3. Security, Privacy & Reliability Requirements

- Passwords are handled only by ASP.NET Core Identity and are never logged or returned.
- Use HTTPS only in production. Secure, HttpOnly, SameSite refresh-token cookie is preferred; keep access tokens short-lived.
- Rotate refresh tokens and revoke token families on suspicious reuse.
- Protect authentication endpoints with rate limiting and generic credential errors.
- Apply strict CORS to the deployed frontend origins.
- Validate file type, size and ownership for uploaded progress photos or custom exercise media.
- Private media must use non-public object keys and signed/authorized retrieval where supported.
- Use optimistic concurrency or conflict handling for active workout/offline sync updates.
- Create automated PostgreSQL backups when the app becomes relied upon; verify restore procedure periodically.
- Health checks must not expose secrets, raw connection strings or personal data.
- Logs must avoid tokens, passwords, body-photo URLs with long-lived credentials, and sensitive request payloads.

3.1 Non-functional requirements

Requirement	Target
Scale	10 normal users; architecture should comfortably tolerate 100 without redesign
Active workout responsiveness	Common set-entry UI updates immediately; network save is asynchronous
API response goal	Typical CRUD reads/writes under ~500 ms excluding cold-start/provider latency
Availability	Best-effort low-cost hosting; active workout should survive temporary connectivity loss
Browser support	Current Chrome/Edge/Safari/Firefox; Android Chrome is primary mobile target
Accessibility	Keyboard reachable controls, proper labels, adequate contrast, no color-only status
Time handling	Store timestamps in UTC; render using user timezone
Units	Persist canonical values where practical; present kg/lb and cm/in according to settings

4. Core Data Model

Entity	Purpose
ApplicationUser	Identity user; display name, status, created timestamp.
UserProfile	Height, preferred units, timezone, optional demographic/profile fields.
Muscle	Canonical muscle/group taxonomy.
Equipment	Canonical equipment taxonomy.
Exercise	Built-in or user-created exercise; instructions, type, equipment.
ExerciseMuscle	Exercise-to-muscle relation with primary/secondary role and optional contribution weight.
Routine	User-owned reusable workout template.
RoutineExercise	Ordered exercise in a routine; rest time, notes, superset group.
RoutineSetTemplate	Target set type, target reps/range, target weight if specified.
WorkoutSession	A started/completed workout with timestamps, title, status and notes.
WorkoutExercise	Snapshot of exercise within a workout; preserves ordering and notes.
WorkoutSet	Weight, reps, RPE, type, completion time, ordering and optional flags.
PersonalRecord	Detected record type/value and source WorkoutSet.
BodyMeasurement	Dated weight/body-fat/body circumference values and notes.
ProgressPhoto	Private dated photo metadata and storage key.
ExerciseNote	Persistent user-specific note shown next time an exercise appears.
WorkoutSchedule	Optional recurring day/routine assignment.
UserSetting	Timers, units, theme, 1RM formula, plate inventory and behavior settings.
RefreshToken	Hashed rotating refresh-token record/device session metadata.

4.1 Key modeling rules

- Workout data is historical and should remain meaningful if a routine or exercise is later edited. Snapshot display names/notes where needed rather than depending only on mutable template values.
- Decimal weight values must support fractional plates (for example 82.5 kg). Do not use floating-point binary types for persisted weights.
- WorkoutSet ordering must be explicit and stable.
- An active WorkoutSession has a lifecycle state such as Active, Completed, Cancelled.
- PersonalRecord rows should be reproducible or safely recomputed from workout data.
- Soft-delete user-created routines/exercises only if necessary for historical references; never break old workout history.

5. API Surface - Initial Contract

/api/auth

- POST /register
- POST /login
- POST /refresh
- POST /logout
- GET /me

/api/exercises

- GET /
- GET /{id}
- POST /
- PUT /{id}
- DELETE /{id}

/api/routines

- GET /
- GET /{id}
- POST /
- PUT /{id}
- DELETE /{id}
- POST /{id}/duplicate

/api/workouts

- GET /
- GET /{id}
- POST /start
- PUT /{id}
- POST /{id}/finish
- POST /{id}/cancel

/api/progress

- GET /exercise/{exerciseId}
- GET /personal-records
- GET /volume
- GET /estimated-one-rep-max

/api/measurements

- GET /

POST /

PUT /{id}

DELETE /{id}

/api/dashboard

GET /summary

/api/export

GET /csv

GET /json

Endpoint naming may evolve, but Codex must preserve consistent REST semantics, authorization, validation, idempotency where appropriate, and documented error shapes.

6. Product Backlog - Epics & User Stories

Priority labels: P0 = required for first usable MVP, P1 = next core/Pro capability, P2 = enhancement, P3 = later/social/operational. Codex should implement dependencies before dependent epics.

Epic 1: Authentication & Accounts [P0]

US-001 Register

As a user, I want to create an account so my workout data is private to me.

Acceptance criteria

- Register with display name, email and password.
- Validate email and password requirements.
- Prevent duplicate normalized emails.
- Create Identity account securely; never store plaintext passwords.
- Establish authenticated session after successful registration.

US-002 Login & session

As a user, I want secure persistent login so I can use the app easily on my phone.

Acceptance criteria

- Email/password login with generic invalid-credential message.
- Short-lived access token plus rotating refresh token.
- Refresh survives browser reload according to chosen secure token strategy.
- Logout revokes current refresh session.

US-003 User profile

As a user, I want to manage my profile and units.

Acceptance criteria

- Edit display name, profile image, optional date of birth/gender, height, current weight.
- Choose kg/lb and cm/in.
- Choose timezone.
- All display values use user preferences.

Epic 2: Exercise Database [P0]

US-010 Exercise library

As a user, I want a searchable exercise library so I can quickly add movements.

Acceptance criteria

- Store name, instructions, primary/secondary muscles, equipment, exercise type/category and optional media URL.
- Seed a useful default library without copyrighted third-party assets.

US-011 Search & filter

As a user, I want instant exercise search and filters.

Acceptance criteria

- Search by exercise name.
- Filter by muscle, equipment and category.
- Results remain responsive on mobile.

US-012 Custom exercises

As a user, I want to create exercises not in the default library.

Acceptance criteria

- Create user-owned custom exercise with name, muscles, equipment, type, notes and optional image.
- Other users cannot edit or view private custom exercises unless sharing is later enabled.

US-013 Exercise details

As a user, I want one exercise page with instructions and my performance context.

Acceptance criteria

- Show instructions, muscles/equipment, media if available, recent performance, PRs and charts entry points.

Epic 3: Workout Routines [P0]

US-020 Create routine

As a user, I want reusable workout templates.

Acceptance criteria

- Set name/description.
- Add ordered exercises.
- Configure set templates, target reps or rep range, optional target weight, rest timer and notes.

US-021 Routine folders

As a user, I want to organize routines into programs/folders.

Acceptance criteria

- Create, rename and delete folders without deleting routines.
- Reorder routines within a folder.

US-022 Edit routine

As a user, I want to maintain routines easily.

Acceptance criteria

- Add/remove/reorder exercises.
- Duplicate exercises.
- Change targets/rest/notes.
- Assign superset group.

US-023 Duplicate routine

As a user, I want to clone a routine before modifying it.

Acceptance criteria

- Duplicate creates an independent user-owned copy with exercises/set templates preserved.

Epic 4: Active Workout [P0]

US-030 Start workout

As a user, I want to start from a routine, previous workout or empty workout.

Acceptance criteria

- Starting creates exactly one active WorkoutSession.
- Show workout name, elapsed duration, exercises, previous values, editable current values and completion controls.
- Warn or offer resume if another active workout exists.

Epic 5: Set Logging [P0]

US-031 Record set

As a user, I want to record weight, reps, RPE and set type.

Acceptance criteria

- Support Normal, Warmup, Drop Set, Failure, AMRAP and Backoff.
- Persist explicit order and completion timestamp.
- Weight supports decimals.

US-032 Previous values

As a user, I want my last performance beside the current set.

Acceptance criteria

- Resolve previous comparable completed workout for that exercise.
- Display previous weight/reps for matching set positions where possible.

US-033 Copy previous

As a user, I want one-tap copying of previous values.

Acceptance criteria

- Copy previous weight/reps into current editable set.
- Do not mark the set complete automatically.

US-034 Manage sets

As a user, I want to add, duplicate, delete and reorder sets.

Acceptance criteria

- Changes update immediately in UI and persist safely.

- Deleting a completed set requires an undo or confirmation pattern.

Epic 6: Rest Timer [P0]

US-040 Automatic timer

As a user, I want a rest timer to start after completing a set.

Acceptance criteria

- Exercise/routine default rest duration.
- Pause, resume, +30s, -30s and skip.
- Sound/vibration/browser notification where permitted.
- Timer continues correctly when tab is backgrounded.

US-041 Exercise-specific rest

As a user, I want different rest periods by exercise.

Acceptance criteria

- RoutineExercise can override global default.
- Timer uses the most specific available setting.

Epic 7: Supersets & Grouped Sets [P1]

US-050 Supersets

As a user, I want to group exercises as supersets, tri-sets or giant sets.

Acceptance criteria

- Visual grouping in routine and active workout.
- Exercise order within group is preserved.
- Rest behavior supports group completion rather than forcing a timer after every individual exercise.

Epic 8: Warm-up Calculator [P1]

US-060 Warmup calculator

As a user, I want warm-up set suggestions based on working weight.

Acceptance criteria

- Generate configurable percentage-based warm-up sets.
- Round to achievable increments based on user setting.
- Suggestions can be inserted into the workout as Warmup set type.
- User can edit or ignore suggestions.

Epic 9: Workout Completion [P0]

US-070 Finish workout

As a user, I want a useful summary after finishing.

Acceptance criteria

- Finish timestamp is stored and session becomes immutable-by-default completed history.
- Show duration, exercises, sets, reps, total volume and newly detected PRs.
- Allow returning to history detail.

Epic 10: Workout History [P0]

US-080 History list

As a user, I want chronological workout history.

Acceptance criteria

- Paginated/infinite list by date.
- Filter by date range, routine and exercise.

US-081 Workout detail

As a user, I want to inspect every exercise and set from a past workout.

Acceptance criteria

- Show workout metadata, exercise order, set values, RPE and notes.

US-082 Edit historical workout

As a user, I want to correct mistakes in old workouts.

Acceptance criteria

- Edits are user-authorized only.
- Derived volume/PRs/charts are recalculated consistently.
- Clearly indicate edit mode to avoid accidental changes.

Epic 11: Exercise History [P0]

US-085 Exercise history

As a user, I want a chronological view of every time I performed an exercise.

Acceptance criteria

- Show date and completed sets per session.
- Link each entry to full workout detail.
- Support pagination and date filters.

Epic 12: Personal Records [P0]

US-090 Automatic PR detection

As a user, I want PRs detected automatically.

Acceptance criteria

- Track highest weight, most reps at a weight, highest estimated 1RM, best set volume and optional workout volume record.
- Do not count incomplete/warmup sets unless configured.

- Historical corrections recompute affected records.

Epic 13: Estimated 1RM [P1]

US-095 Estimated one-rep max

As a user, I want estimated 1RM to track strength trends.

Acceptance criteria

- Support Epley, Brzycki and Lombardi formulas.
- Default formula is configurable per user.
- Avoid nonsensical calculations for invalid rep ranges.

Epic 14: Progress Charts [P1]

US-098 Exercise charts

As a user, I want charts for strength and training volume.

Acceptance criteria

- Charts for best weight, estimated 1RM, volume and max reps.
- Ranges: 1 month, 3 months, 6 months, 1 year, all time.
- Charts work on mobile and include accessible textual values.

Epic 15: Body Measurements [P1]

US-100 Body weight

As a user, I want dated weight tracking.

Acceptance criteria

- Date, weight and note.
- Weight trend chart.

US-101 Measurements

As a user, I want to track body measurements.

Acceptance criteria

- Support body fat %, chest, waist, hips, arms, thighs, calves, shoulders and neck.
- Partial entries allowed; not every metric is required.

US-102 Progress photos

As a user, I want private front/side/back progress photos.

Acceptance criteria

- Store date, optional weight, pose/type and private storage key.
- Only owner can retrieve/delete.
- Enforce upload size/type limits.

Epic 16: Muscle Heatmap [P2]

US-110 Muscle contribution summary

As a user, I want to see which muscle groups a workout trained.

Acceptance criteria

- Compute from exercise-muscle mappings and completed sets.
- Primary and secondary contribution weights are configurable in data model.
- Show ranked textual summary even if body heatmap visualization is not implemented initially.

Epic 17: Training Statistics [P1]

US-120 Training stats

As a user, I want weekly/monthly training statistics.

Acceptance criteria

- Workouts, training time, total volume, sets, exercises and PR count.
- Current streak uses documented definition.
- Stats exclude cancelled workouts.

Epic 18: Calendar [P2]

US-130 Calendar history

As a user, I want workouts shown on a calendar.

Acceptance criteria

- Month view marks completed workouts.
- Tap date to view one or more workouts.
- Timezone-correct date grouping.

Epic 19: Workout Scheduling [P2]

US-140 Routine schedule

As a user, I want to assign routines to days.

Acceptance criteria

- Configure day-of-week routine schedule.
- Dashboard can surface next planned workout.
- Schedule does not create a workout until user starts it.

Epic 20: Notes [P1]

US-150 Notes

As a user, I want workout, exercise and set notes.

Acceptance criteria

- Workout notes persist in history.
- Persistent exercise note can automatically appear next time.
- Set-level note is optional and historical.

Epic 21: Plate Calculator [P2]

US-160 Plate calculator

As a user, I want to know which plates to load.

Acceptance criteria

- Configurable bar weight and available plate inventory.
- Compute plates per side for target weight.
- Handle impossible exact target and show nearest achievable options.

Epic 22: Progressive Overload Suggestions [P2]

US-170 Suggestions

As a user, I want optional next-workout load suggestions.

Acceptance criteria

- Base suggestion on configured rep range and recent completed work sets.
- Never change workout weights automatically.
- Show rationale such as all sets reached top of target range.
- Allow user to dismiss/override.

Epic 23: Workout Goals [P2]

US-180 Goal preference

As a user, I want to choose a training goal.

Acceptance criteria

- Options include Fat Loss, Strength, Hypertrophy, General Fitness and Endurance.
- Goal is descriptive initially and must not silently rewrite programs.

Epic 24: Dashboard [P1]

US-190 Dashboard

As a user, I want the home screen to show what matters now.

Acceptance criteria

- Prominent Start/Resume Workout action.
- Next scheduled routine if configured.
- Weekly summary, recent PRs and latest body-weight trend.
- Dashboard loads with a small number of aggregated API calls.

Epic 25: Data Export [P1]

US-200 Export

As a user, I want complete ownership of my data.

Acceptance criteria

- CSV export for workouts, workout sets, exercises and body measurements.
- JSON export of all user-owned data with schema/version metadata.
- Only current user data is included.

Epic 26: Data Import [P2]

US-210 Import

As a user, I want to import compatible CSV data.

Acceptance criteria

- Upload, parse and preview before commit.
- Validate required columns and show row-level errors.
- Import is transactional or safely resumable.
- Prevent accidental duplicate import when feasible.

Epic 27: Theme [P0]

US-220 Theme

As a user, I want Light, Dark and System themes.

Acceptance criteria

- Dark is first-class and optimized for gym use.
- Preference persists per user/device as appropriate.

Epic 28: Mobile Experience [P0]

US-230 Mobile workout UX

As a user, I want large, fast controls between sets.

Acceptance criteria

- Primary workout actions reachable with one hand.
- Numeric inputs open appropriate mobile keyboard.
- No horizontal scrolling for standard set entry.
- Accidental navigation away from an unsaved/active workout is handled safely.

Epic 29: Progressive Web App [P1]

US-240 Installable PWA

As a user, I want to install the site like an app.

Acceptance criteria

- Manifest, icons, standalone display mode and service worker.
- Cache static app shell and safe read-only reference data.
- Browser notifications are opt-in.

Epic 30: Offline Workout Mode [P1]

US-250 Offline active workout

As a user, I want to keep logging when connectivity is poor.

Acceptance criteria

- Active workout and pending mutations are persisted in IndexedDB.
- UI clearly shows offline/sync status.
- Reconnect flushes mutations in deterministic order.
- Conflicts never silently overwrite newer server data.

Epic 31: Multiple Users & Isolation [P0]

US-260 User isolation

As an operator, I need strict separation between users.

Acceptance criteria

- Every user-owned aggregate is bound to owner UserId.
- API authorization uses authenticated identity, not client-supplied ownership.
- Integration tests attempt cross-user access and expect 404/403 according to policy.

Epic 32: Friends & Sharing - Later [P3]

US-270 Friends/sharing

As a user, I may want to share selected training items.

Acceptance criteria

- Optional later phase; default everything private.
- Share workout, routine or PR with explicit privacy setting.
- Revocable link/friend access.

Epic 33: Copy Shared Routine - Later [P3]

US-280 Copy shared routine

As a user, I want my own editable copy of a shared routine.

Acceptance criteria

- Copy creates independent ownership.
- No future edits leak between original and copy.

Epic 34: Administration [P2]

US-290 Admin panel

As the owner, I want basic operational controls.

Acceptance criteria

- List users, disable/enable account, manage default exercise catalog, view app version/health and sanitized errors.
- Admins never see passwords or unrestricted private progress photos by default.
- Admin actions are authorized and auditable.

Epic 35: Backups & Recovery [P1]

US-300 Backup/recovery

As the operator, I want recoverable data.

Acceptance criteria

- Document automated PostgreSQL backup strategy.
- Keep periodic backup copies according to provider limits/budget.
- Document and test restore procedure.
- User JSON export is not a substitute for database backup.

7. Core Business Rules & Calculations

7.1 Volume

Default set volume = weight x repetitions for completed resistance-training sets. Workout volume is the sum of eligible set volumes. Bodyweight-only movements may use reps-only metrics unless the user adds external load; do not invent bodyweight load into strength volume without an explicit rule.

7.2 Personal records

PR detection must run from completed eligible work sets and be deterministic. Warmups should be excluded by default. Historical edits can invalidate prior PRs, so the system needs a recomputation path rather than relying only on append-only celebratory events.

7.3 Estimated 1RM

Epley: 1RM = weight * (1 + reps / 30)

Brzycki: 1RM = weight * 36 / (37 - reps)

Lombardi:1RM = weight * reps^0.10

Use sensible rep limits and validation. Exact 1-rep sets can simply use the lifted weight.

7.4 Progressive overload

Initial recommendation rule can be deliberately simple: when all prescribed work sets reach the top of the target rep range with acceptable RPE, suggest the next configured load increment; otherwise suggest maintaining load. This is guidance only and should be transparent to the user.

7.5 Time and duration

Store UTC timestamps. Workout duration is based on started/completed timestamps but may later support paused time. The rest timer should use an absolute end timestamp rather than decrement-only state so background tabs remain accurate.

8. Required Screens

Login / Register

Minimal authentication flows.

Dashboard

Start/resume workout, next routine, weekly stats, recent PRs and latest weight.

Routine Library

Folders/programs, create/edit/duplicate routine.

Routine Editor

Exercise search, drag/reorder, set targets, notes, rest and superset grouping.

Exercise Library

Search/filter and custom exercise creation.

Active Workout

Highest-priority mobile screen: previous vs current sets, RPE, notes, completion, timer, add set.

Workout Summary

Duration, volume, sets/reps and PR celebration.

Workout History

Chronological history and filters.

Workout Detail

Historical exercise/set values and edit action.

Exercise Detail

Instructions, history, PRs and progress charts.

Progress

Charts, training stats and body measurements.

Calendar

Workout history and optional schedule.

Settings

Units, timezone, theme, timer behavior, 1RM formula, plates and account actions.

Admin

Users, default exercises, health/version and operational tools.

8.1 Active workout wireframe intent

BENCH PRESS

Previous	kg	reps	RPE	done
80 x 8	[82.5]	[8]	[8]	[ ]
80 x 8	[82.5]	[8]	[ ]	[ ]
80 x 7	[82.5]	[ ]	[ ]	[ ]

[+ ADD SET] Rest 01:27

Exercise note: Seat position 4. Neutral grip.

- Keep previous performance visually available without extra navigation.
- Make completion checkbox/button large.
- Use numeric input modes on mobile.
- Preserve scroll position and active timer.
- A failed network save must not erase what the user just entered.

9. Deployment & Low-Cost Infrastructure

Provider free tiers change. Treat the following as the preferred architecture rather than a permanent pricing guarantee; verify provider limits before deployment.

Concern	Preferred option	Notes
Source control / CI	Private GitHub repository + GitHub Actions	Build/test on pull request and deploy from main.
Vue frontend	Cloudflare Pages	Static SPA/PWA hosting, custom domain and HTTPS.
ASP.NET API	Railway container or another low-cost container host	Use Dockerfile; avoid hosts with disruptive sleep if daily gym use matters.
PostgreSQL	Neon PostgreSQL	Good fit for tiny workload; use pooled connection string if provider recommends it.
Object storage	Cloudflare R2 / Supabase Storage / equivalent	Only needed when progress photos/custom images are implemented.
DNS / TLS	Cloudflare	Automatic HTTPS and DNS management.
Secrets	Hosting provider secret/environment variables	Never commit connection strings, JWT secrets or storage credentials.

9.1 Local development

```
docker compose up -d postgres

# API: dotnet run --project src/WorkoutTracker.Api

# Web: npm install && npm run dev --prefix src/WorkoutTracker.Web
```

Codex should provide a docker-compose.yml for PostgreSQL and a sample .env.example without real secrets. README must document initial migration and local startup commands.

10. Implementation Plan for Codex

Phase 0 - Foundation

- Create solution/projects and Vue app.
- Configure PostgreSQL, EF Core, migrations, Identity, OpenAPI, structured errors, Serilog, health checks.
- Set up CI build/test.

Phase 1 - Usable MVP

- Epics 1-6, 9-12, 27-28, 31.
- Seed exercise library.
- Deliver full routine -> active workout -> finish -> history loop.

Phase 2 - Progress & Pro

- Epics 7-8, 13-15, 17, 20, 24-25, 35.
- Add charts, measurements, notes, export and backup documentation.

Phase 3 - PWA & Resilience

- Epics 29-30.
- Add installable PWA, IndexedDB active-workout cache/outbox and sync conflict handling.

Phase 4 - Advanced Training

- Epics 16, 18-23, 26.
- Heatmap, calendar/scheduling, plate calculator, progressive overload suggestions, goals and import.

Phase 5 - Operations / Optional Social

- Epic 34 then optional Epics 32-33.
- Keep social sharing disabled until explicit privacy model is implemented and tested.

10.1 Definition of Done for every Codex task

- Application builds with zero compiler errors.
- Relevant unit/integration tests are added and passing.
- Any schema change includes an EF Core migration.
- Authorization and validation are tested for new user-owned resources.
- OpenAPI remains valid.
- Frontend handles loading, empty and error states.
- Mobile layout is checked at a narrow viewport.
- No secrets or production credentials are committed.
- README or specification notes are updated when setup/behavior changes.

11. Master Prompt to Give Codex

Use the following as the persistent instruction at the start of the repository or Codex session. Then provide one phase/epic at a time.

> You are implementing WorkoutTracker, a production-quality private workout tracking PWA for approximately 10 users.
>
> STACK
> - .NET 10 / ASP.NET Core 10 Web API
> - Entity Framework Core 10
> - PostgreSQL
> - ASP.NET Core Identity
> - JWT access tokens + rotating refresh tokens
> - Vue 3 + TypeScript + Vite
> - Pinia + Vue Router + Tailwind CSS
> - Chart.js for charts
>
> ARCHITECTURE
> - WorkoutTracker.Domain
> - WorkoutTracker.Application
> - WorkoutTracker.Infrastructure
> - WorkoutTracker.Api
> - WorkoutTracker.Web
>
> NON-NEGOTIABLE RULES
> 1. Backend owns business rules and authorization.
> 2. Never trust client-supplied UserId for ownership.
> 3. All user data is isolated by authenticated UserId.
> 4. Use DTOs; never return EF entities directly.
> 5. Use async I/O, validation, DI, nullable reference types and EF migrations.
> 6. Vue uses TypeScript, Composition API and <script setup>.
> 7. Design mobile-first; active workout logging is the primary UX.
> 8. Add tests with every feature, including cross-user authorization tests.
> 9. Keep the app runnable after every change.
> 10. Do not introduce paid dependencies or copy proprietary Strong assets/designs.
> 11. If this specification conflicts with an implementation shortcut, follow the specification.
>
> WORKFLOW
> - Before coding an epic, briefly identify affected entities, endpoints, migrations, components and tests.
> - Implement the smallest coherent vertical slice.
> - Run build/tests.

- Report files changed, migration created, tests added and any known follow-up work.

Use the attached Product & Engineering Specification as the source of truth for all epics and acceptance criteria.

12. Testing Strategy

Test type	Minimum coverage
Domain/unit	PR calculations, 1RM formulas, volume, plate calculator, overload suggestion rules, timer helpers.
API integration	Auth lifecycle, CRUD ownership, routine creation, workout start/finish, historical edits, export, admin authorization.
Security integration	User A cannot read/update/delete User B resources; disabled users are blocked as designed.
Frontend unit/component	Active set row, previous-value copy, rest timer state, routine editor operations, validation states.
E2E smoke	Register/login -> create routine -> start workout -> log sets -> finish -> see history and PR.
Offline later	Create/update active workout offline -> reconnect -> sync exactly once without data loss.

12.1 Seed data

Seed muscles, equipment and a curated default exercise library. Seed data should use stable identifiers or safe upsert logic so migrations/deployments do not create duplicates. Do not seed real user accounts or secrets into production.

13. Decisions, Risks & Explicit Non-Goals

13.1 Decisions

- REST over GraphQL for simplicity.
- PostgreSQL as the only production relational database.
- ASP.NET Identity instead of outsourcing auth initially.
- PWA before native Android/iOS app.
- Suggestions are rule-based first; no AI dependency required for core product.
- Private-by-default sharing model.

13.2 Risks to avoid

- Trying to build all 35 epics in one Codex prompt.
- Overengineering microservices for a 10-user app.
- Making the frontend responsible for authorization.
- Using unstable free hosting that loses or expires the database.
- Implementing offline sync without deterministic IDs/versioning/conflict behavior.
- Storing progress photos publicly.
- Treating PR events as irreversible instead of recomputable.

13.3 Non-goals for initial launch

- Public social network/feed.
- Payments or subscriptions.
- Coach marketplace.
- Native mobile apps.
- Nutrition/calorie tracking.
- Medical advice or injury diagnosis.
- Automatic program changes without user approval.

14. Release Checklist for First Real Use

- Production database has backups configured or a documented manual backup cadence.
- HTTPS and production CORS are correct.
- Registration can be disabled/invite-only if desired for the 10-person group.
- Cross-user integration tests pass.
- Refresh-token and logout behavior tested on phone.
- Active workout survives refresh and temporary network loss at least at the current implementation level.
- Export works and produces readable data.
- Database migrations run automatically or via a documented safe release step.
- Privacy policy/short notice explains what data is stored, especially progress photos.
- Admin account is protected with a strong unique password.